

Navigatore Burocratico — Analisi Tecnica

Versione documento: 0.4

Data: 2026-03-27

Stato: Draft

1. Vision & Scope

Navigatore Burocratico è un'applicazione mobile-first che guida cittadini e professionisti italiani attraverso le procedure burocratiche passo per passo.

MVP: Regione Calabria — Provincia di Cosenza.

L'obiettivo è abbattere la complessità della burocrazia italiana fornendo:

- Percorsi guidati per ogni pratica (es. DIA, SCIA, permesso di costruire, successioni, ecc.)
- Lista documenti necessari per ogni step
- Indicazioni su uffici competenti, costi, tempi stimati
- Assistenza intelligente tramite LLM locale (Ollama)

2. Stack Tecnologico

2.1 Backend

Componente	Tecnologia	Versione
Linguaggio	Python	3.13
Framework API	FastAPI	≥ 0.115
ORM	SQLAlchemy	≥ 2.0
Migrazioni DB	Alembic	≥ 1.14
Server ASGI	Uvicorn	≥ 0.30
Configurazione	Pydantic Settings	≥ 2.7
HTTP client	HTTPX	≥ 0.28
Autenticazione	PyJWT + bcrypt	≥ 2.9 / ≥ 4.0
AI / LLM	Ollama (self-hosted)	—

2.2 Mobile

Componente	Tecnologia
Framework	Flutter

Componente	Tecnologia
Target	iOS + Android

2.3 Infrastruttura (pianificata)

Componente	Tecnologia
Database	PostgreSQL (prod + dev via Docker)
Container	Docker + Docker Compose
CI/CD	GitHub Actions
Deploy backend	TBD (VPS / Railway / Render)

3. Architettura del Backend

3.1 Struttura attuale

```
backend/
├── api/
│   ├── deps.py          # Dipendenza get_current_user (JWT)
│   └── v1/
│       ├── auth.py      # register, token, refresh, me
│       ├── chat.py      # POST /v1/chat (Ollama)
│       ├── procedures.py # GET /v1/procedures, /{slug}, /{slug}/steps
│       └── progress.py   # GET/PUT /v1/progress/{slug}
├── core/
│   ├── config.py        # Settings via pydantic-settings
│   ├── rate_limit.py    # Sliding window in-memory
│   ├── db/              # Session SQLAlchemy async
│   ├── models/          # ORM: Procedure, Step, Document, User, UserProgress
│   ├── schemas/         # Pydantic response schemas
│   ├── services/        # ai_service, auth_service, procedure_service
│   └── main.py           # Entry point FastAPI
```

3.2 Struttura dati procedure

```
data/
├── procedures/
│   ├── calabria/
│   │   └── cosenza/ # File JSON/YAML delle procedure
```

3.3 Flusso architetturale

```
App Flutter
└── ▼ HTTPS (REST JSON)
    FastAPI Backend
    ├── api/v1/      → Route handlers (thin layer)
    ├── services/    → Business logic
    ├── models/      → ORM / schema Pydantic
    ├── db/          → PostgreSQL
    └──
        ├── Procedure definitions (JSON/YAML su disco o in DB)
        └── Ollama (HTTP interno) → LLM locale per assistenza
```

4. Variabili d'Ambiente

Variabile	Descrizione
APP_ENV	development / production
SECRET_KEY	Chiave per JWT / sessioni
DATABASE_URL	URI connessione al DB
OLLAMA_BASE_URL	URL base istanza Ollama (es. <code>http://localhost:11434</code>)
OLLAMA_MODEL	Nome modello Ollama (es. <code>llama3</code> , <code>mistral</code>)
POSTGRES_USER	Utente PostgreSQL (Docker Compose)
POSTGRES_PASSWORD	Password PostgreSQL (Docker Compose)

5. Roadmap

Fase 0 — Fondamenta (completata)

- Scaffold backend (FastAPI, SQLAlchemy, Alembic, config)
- Struttura cartelle procedure (calabria/cosenza)
- Setup Docker Compose (backend + PostgreSQL)
- CI GitHub Actions (lint + test)
- Prima migrazione Alembic (placeholder vuoto `a1b2c3d4e5f6` creato — schema reale definito in Fase 1)

Fase 1 — Procedure Core (completata)

- Schema JSON per una procedura (step, documenti, ufficio, tempi, costi)
- Prime procedure Cosenza: SCIA Edilizia (7 step), Permesso di Costruire (10 step)
- Modello ORM: `Procedure`, `Step`, `Document`
- API REST: `GET /v1/procedures`, `GET /v1/procedures/{slug}`, `GET /v1/procedures/{slug}/steps`
- Service JSON-backed (no DB richiesto in CI)

Fase 2 — Assistente AI (completata)

- `AIService`: client HTTPX asincrono verso Ollama `/api/chat`
- `POST /v1/chat` context-aware: system prompt arricchito con dati procedura attiva
- Prompt engineering focalizzato su burocrazia Cosenza, con "non so" esplicito
- Rate limiting sliding window 10 req/min per IP (in-memory, no dipendenze esterne)
- Gestione errori: `ConnectError`, `Timeout`, `HTTPStatusError` → HTTP 503

Fase 3 — Autenticazione & Utenti (completata)

- Modelli `User` e `UserProgress` (timezone-aware, cascade delete)
- JWT: access token (30 min) + refresh token (30 giorni), `type` claim per distinguerli
- `POST /v1/auth/register`, `/token` (OAuth2), `/refresh`; `GET /v1/auth/me`
- `GET/PUT /v1/progress/{slug}` — salvataggio step completati per utente autenticato
- Test con SQLite in-memory isolato per fixture (conftest.py)

Fase 4 — App Flutter (MVP)

- Setup progetto Flutter
- Schermata home: lista categorie procedure
- Schermata procedura: step-by-step con checklist documenti
- Schermata chat con assistente AI
- Gestione stato offline (cache locale procedure)

Fase 5 — Qualità & Deploy

- Test di integrazione API (pytest + httpx AsyncClient)
- Dockerizzazione completa (multi-stage build)
- Deploy backend su server (VPS o PaaS)
- Deploy app Flutter su store (TestFlight / Play Console beta)

Backlog / Future estensioni

- Supporto altre province calabresi
 - Supporto altre regioni italiane
 - Notifiche push (scadenze pratiche, aggiornamenti normative)
 - Upload e gestione documenti utente (storage S3-compatible)
 - Integrazione con servizi PA (SUAP, sportello unico, ecc.)
 - Versione web (PWA o React/Next.js)
-

6. Funzionalità Pianificate — Dettaglio

6.1 Catalogo Procedure

Ogni procedura è descritta da:

- **Nome e categoria** (edilizia, successioni, licenze commerciali, ecc.)
- **Ente competente** (Comune, Provincia, Regione, SUAP, ecc.)
- **Step sequenziali** numerati, ognuno con:
 - Descrizione azione
 - Documenti necessari (nome, obbligatorio/opzionale, note)
 - Ufficio / sportello di riferimento
 - Costo stimato
 - Tempo stimato
- **Tag** per filtraggio (es. **edilizia**, **privato**, **impresa**)

6.2 Assistente AI (Ollama)

- Risponde a domande contestuali sulla procedura attiva
- Accesso solo a knowledge base locale (nessuna chiamata esterna)
- Gestione "non so" con rimando a fonti ufficiali
- Possibile RAG su PDF normativi locali

6.3 Progresso Utente

- Salvataggio step completati per pratica
- Checklist documenti con spunta

- Riepilogo stato ("hai completato 3/7 step")

7. Bug Noti / Rischi Tecnici

ID	Tipo	Descrizione	Priorità
RISK-01	Dati	Le procedure burocratiche variano per comune e cambiano frequentemente — necessario processo di aggiornamento dati	Alta
RISK-02	AI	Ollama richiede hardware locale adeguato in produzione (GPU o CPU potente) — valutare alternativa cloud LLM	Alta
RISK-03	Conformità	Informazioni burocratiche errate possono causare problemi reali all'utente — disclaimer + revisione umana	Alta
RISK-04	Offline	App Flutter deve funzionare parzialmente offline — strategia cache da definire	Media
RISK-05	Auth	JWT secret key management in produzione — usare secrets manager	Media
RISK-06	CI	Il job <code>test</code> imposta <code>DATABASE_URL</code> senza avviare Postgres — risolto in fase 3 usando SQLite in-memory per i test DB-backed	Chiuso
RISK-07	Deploy	Il <code>Dockerfile</code> avvia direttamente uvicorn senza eseguire <code>alembic upgrade head</code> — lo schema DB non viene creato automaticamente all'avvio del container	Media

8. Convenzioni di Sviluppo

- **Branch:** `main` (stabile) → PR da branch `fase-N`; `docs` per documentazione
- **Commit:** Conventional Commits (`feat:`, `fix:`, `chore:`, `docs:`)
- **Linting:** ruff (configurato in `pyproject.toml`, regole E/F/I, line-length 100); dart format per Flutter (TBD)
- **Test:** pytest + pytest-asyncio per backend; flutter test per mobile
- **Lingua codice:** inglese (nomi variabili, commenti, commit); italiano per documentazione utente e dati procedure